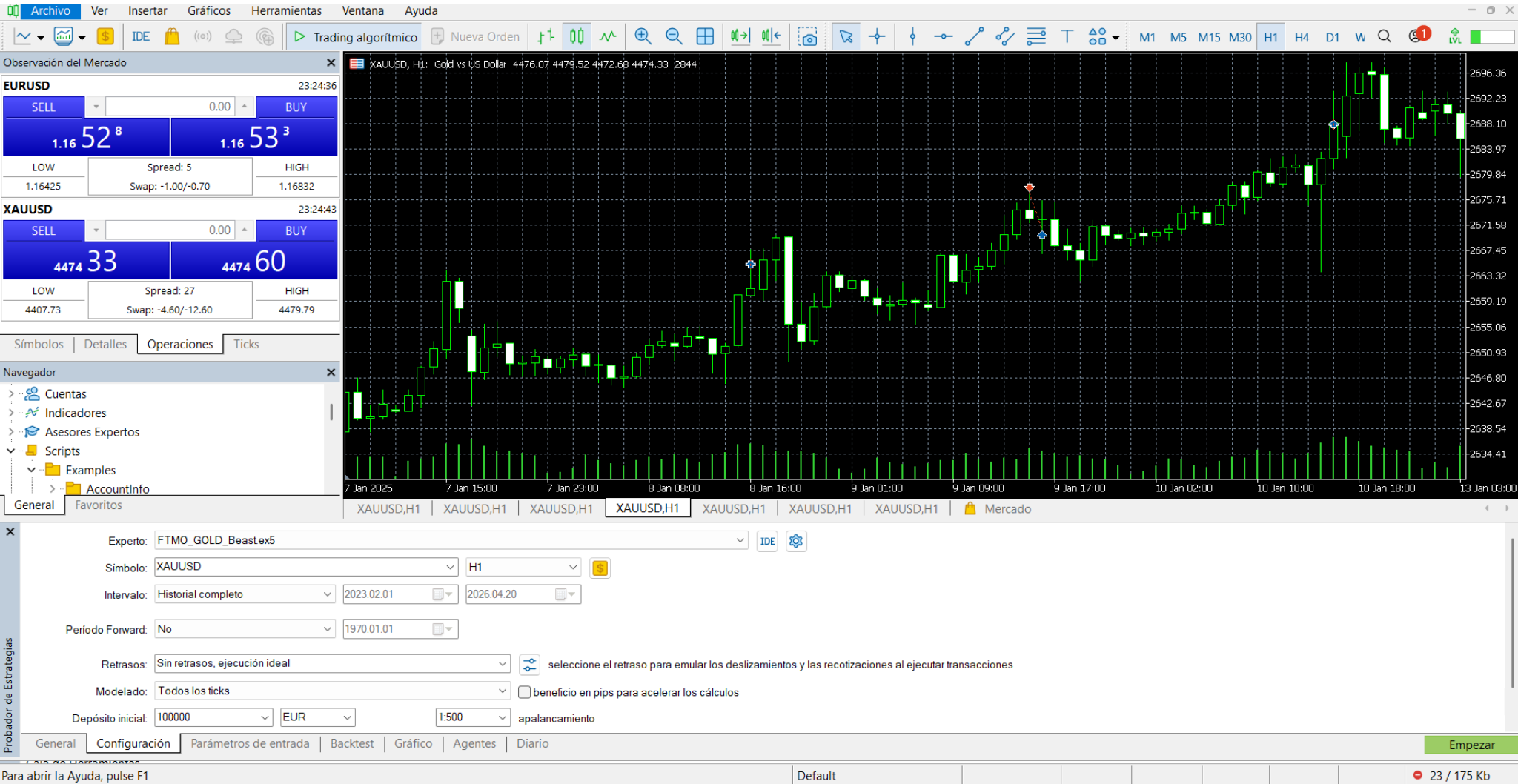


MANUAL QUANT: TRADING PARA INFORMÁTICOS

Mapeo de la Arquitectura de MetaTrader 5 a Estructuras de Software e IA



Escenario 1: Observación del Mercado (Data Streaming)

Este panel es el socket de datos en tiempo real. Para la IA, es la fuente de la verdad donde se reciben las cotizaciones tick a tick.

Símbolo (Ej. XAUUSD)

El identificador del instrumento financiero (Oro vs Dólar). Es el cruce de dos valores.

 **Traducción CS:** Es el `Stream_ID` o `Endpoint_Ticker` que le pasamos a la API para suscribirnos a un Websocket de datos.

BID (Vender) / ASK (Comprar)

Los dos precios del mercado. Bid es el precio al que el broker te compra a ti. Ask es al que te vende. Siempre compras caro y vendes barato.

 **Traducción CS:** Variables float `bid_price`, `ask_price`; La IA no predice "un precio", predice la curva promedio entre ambos.

XAUUSD, Gold vs US Dollar	
• Tamaño de tick	0.01
• Precio del tick	0.1
➤ Bid	4474.33
➤ Bid High	4479.52
➤ Bid Low	4407.73
➤ Ask	4474.60
➤ Ask High	4479.79
➤ Ask Low	4407.87
➤ Cambio diario	0.42%
• Precio de apertura	4462.01
• Precio de cierre	4455.79

Spread

La diferencia exacta entre el Bid y el Ask. Es la comisión del broker.

 **Traducción CS:** Es el "Penalty" o coste computacional por transacción. Tu modelo predictivo debe generar un *Alpha* (retorno esperado) mayor al *Spread* para no perder dinero matemático.

EURUSD			23:24:36			XAUUSD			23:24:43		
SELL	0.00		BUY			SELL	0.00		BUY		
1.16	52 ⁸	1.16	53 ³			4474	33	4474	60		
LOW	Spread: 5		HIGH			LOW	Spread: 27		HIGH		
1.16425	Swap: -1.00/-0.70		1.16832			4407.73	Swap: -4.60/-12.60		4479.79		

Escenario 2: Navegador (El Sistema de Archivos)

Es el explorador de archivos donde MetaTrader monta (mounts) los scripts compilados que van a interactuar con los gráficos.

Asesores Expertos (EA)

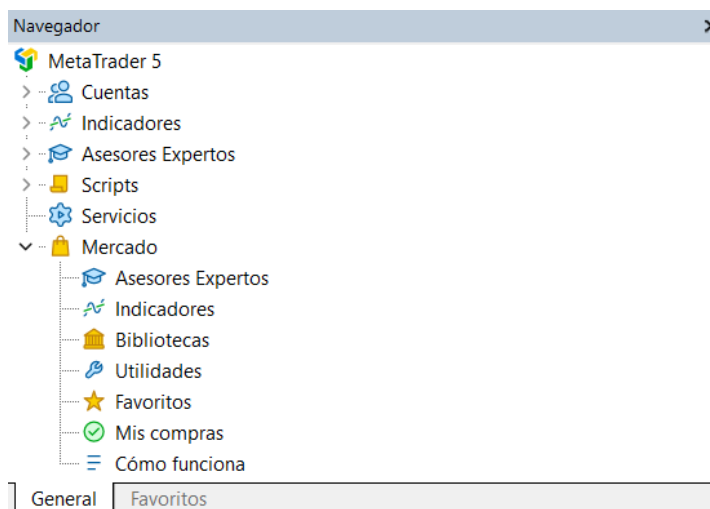
Los robots de trading clásicos programados en lenguaje MQL5 (un derivado de C++).

 **Traducción CS: Son *Daemons* o procesos en segundo plano ligados a un hilo (Thread) del gráfico. Se ejecutan continuamente en cada actualización del socket (OnTick).**

Scripts

Programas de una sola ejecución. Aquí es donde vivirá nuestro código Python si lo lanzamos desde la plataforma.

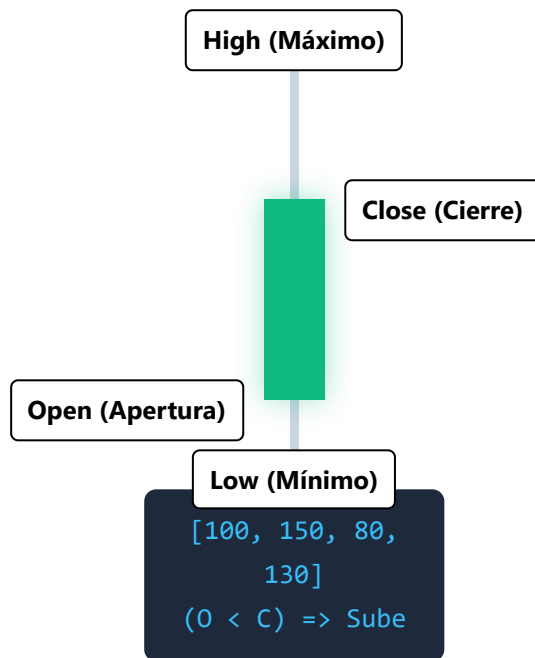
 **Traducción CS: Procesos procedurales de ejecución lineal (Run-once). Hacen una tarea (ej. extraer un CSV, cerrar todas las órdenes) y el proceso muere (Terminate).**



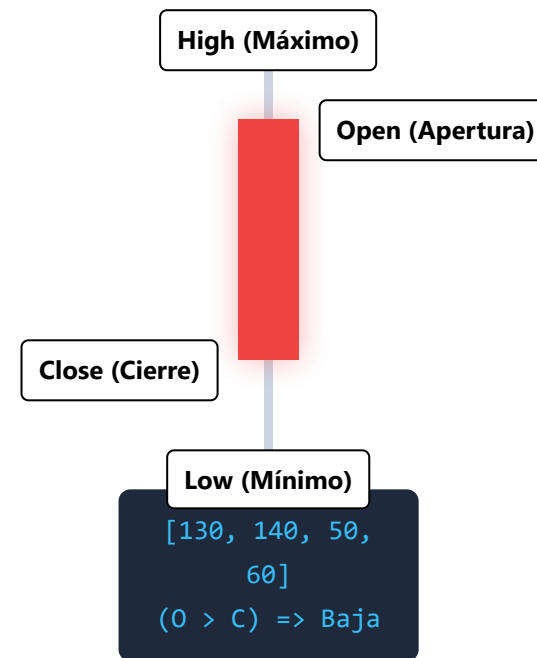
Escenario 3: El Gráfico (Estructura de Datos OHLC)

Un informático ve un gráfico de líneas como un Array 1D. Sin embargo, el mercado esconde volatilidad intradía. Para que una red neuronal o un Random Forest tenga contexto, usamos Velas Japonesas (Matriz 2D).

Vela Alcista (Bullish)



Vela Bajista (Bearish)



 **Traducción CS:** Cada vela no es un dato entero, es un objeto JSON o una fila en un DataFrame de Pandas con 4 columnas: **Open**, **High**, **Low**, **Close**. El tamaño de la vela (High - Low) es nuestra principal variable de **Varianza / Ruido** para entrenar la IA.



Escenario 4: Probador de Estrategias (El Laboratorio QA)

Este es el motor de simulación histórico. Toma tu bot, lo somete a datos del pasado y comprueba si sobrevive o quiebra la cuenta. Aquí te explicamos cada parámetro que sale en esa ventana.

Experto / Símbolo / Intervalo

Qué archivo ejecutar, en qué mercado (Ej. XAUUSD) y el bloque de fechas (Ej. 01/04/2026 - 20/04/2026).

 **CS:** Payload_Binary, Target_DB_Table, y parámetros Start_Date / End_Date para el query SQL (Big Data) de la simulación.

Período Forward (Out-of-Sample)

Divide la prueba. Optimiza el bot en la primera mitad de los datos, y luego lo prueba a ciegas en la segunda mitad.

 **CS:** Es la técnica de *Train-Test Split* o *Cross-Validation* en Machine Learning. Evita el "Overfitting" (sobreajuste).

GeneralFavoritos

XAUUSD,H1XAUUSD,H1XAUUSD,H1XAUUSD,H1XAUUSD,H1XAUUSD,H1XAUUSD,H1

X

General

Probador de Estrategias

Experto:FTMO_GOLD_Beastex5

Simbolo:XAUUSD

Intervalo:Historial completo

Periodo Forward:No

Retrasos:Sin retrasos, ejecución ideal

Modelado:Todos los ticks

Depósito inicial:100000

Optimización:Deshabilitado

H1

\$

2023.02.01

2026.04.20

1970.01.01

seleccione el retraso para emular los deslizamientos y las recotizaciones al ejecutar transacciones

beneficio en pips para acelerar los cálculos

apalancamiento

modo visual con representación de gráficos, indicadores y comercio

Retrasos (Latencia de Ejecución)

Simula que internet va lento o que el servidor del broker tarda en procesar tu orden. Puede ser 0 ms, 50 ms, o aleatorio.

 **CS:** Inyección de latencia de red asíncrona. Si tu código asume respuesta instantánea del servidor (0 ms ping), fallará en producción. Evalúa la tolerancia a fallos.

Modelado (Todos los ticks vs OHLC)

Cómo simula el precio. "Todos los ticks" recrea cada movimiento microscópico. "OHLC a 1 minuto" es una aproximación para ir más rápido.

 **CS:** Granularidad del bucle de simulación. *Todos los ticks* gasta muchísima RAM y CPU, pero garantiza precisión absoluta. *OHLC* comprime los datos para ahorrar recursos.

Depósito inicial y Apalancamiento

Dinero virtual (Ej. 100,000) y poder de compra (1:500 significa que con 1€ controlas 500€ en el mercado).

 **CS:** Estado de las variables iniciales (`initial_balance = 100000`). El apalancamiento es un multiplicador de riesgo matemático. Si el apalancamiento es alto, el *Float Overflow* (quiebra) de la cuenta ocurre más rápido.

Optimización

Permite correr el bot miles de veces probando diferentes combinaciones de indicadores para encontrar la más rentable.

 **CS:** Es la fase de *Hyperparameter Tuning*. MetaTrader utiliza Algoritmos Genéticos (Fast Optimization) o Búsqueda en Cuadrícula (Grid Search) para encontrar los pesos óptimos de las variables.

5. Arquitectura del Código en Python

Conociendo la teoría, así es como estructuramos el código (el "Script") para que se conecte a la API de MT5, extraiga los datos OHLC y entrene una IA validada (reemplazando el botón manual del Probador).

```
import MetaTrader5 as mt5
import pandas as pd
import numpy as np
from sklearn.ensemble import RandomForestClassifier

# 1. API: Conexión y Extracción de Datos OHLC (Gráfico de Velas)
mt5.initialize()
# Solicitamos 10,000 tuplas OHLC del servidor
velas = mt5.copy_rates_from_pos("XAUUSD", mt5.TIMEFRAME_H1, 0, 10000)
df = pd.DataFrame(velas)
df['time'] = pd.to_datetime(df['time'], unit='s')

# 2. FEATURE ENGINEERING: Traduciendo finanzas a métricas IA
# Varianza del mercado usando las colas de la vela (High - Low)
df['Varianza_Vela'] = df['high'] - df['low']
# Tasa de cambio (Retorno) usando el Close
df['Retorno'] = df['close'].pct_change()

# 3. CREACIÓN DEL TARGET (Supervisado)
# Si el cierre futuro es mayor, clase 1 (Comprar). Si es menor, -1 (Vender)
df['Target'] = np.where(df['close'].shift(-1) > df['close'], 1, -1)
df.dropna(inplace=True)

# 4. CROSS-VALIDATION (Equivalente informático al 'Período Forward')
# Train: 80% inicial | Test: 20% final
split_idx = int(len(df) * 0.8)
X = df[['Varianza_Vela', 'Retorno']]
```

```
Y = df['Target']

X_train, X_test = X.iloc[:split_idx], X.iloc[split_idx:]
Y_train, Y_test = Y.iloc[:split_idx], Y.iloc[split_idx:]

# 5. ENTRENAMIENTO Y TESTEO
modelo_ia = RandomForestClassifier(n_estimators=100)
modelo_ia.fit(X_train, Y_train)

precision = modelo_ia.score(X_test, Y_test)
print(f"Exactitud predictiva en Test (Out of Sample): {precision * 100:.2f}%")
```

Escenario 6: Parámetros de Entrada (Hyperparameter Tuning)

Esta pestaña permite configurar las variables internas del bot. Cuando realizas una "Optimización", MetaTrader itera sobre estos valores para encontrar la combinación que genera más dinero.

-  **Traducción CS:** Esto es una interfaz gráfica para **Grid Search (Búsqueda en Cuadrícula)** o algoritmos genéticos .
- **Valor:** Valor por defecto (Default state).
 - **Empezar / Paso / Parar:** Define el bucle iterativo `for i in range(empezar, parar, paso)` para optimizar hiperparámetros.

Categoría	Parámetros Clave	Significado para el Informático
1. STRATEGY	FastMA, RSI_Period, ATR_Period	Tamaño de la "ventana" (Window Size) para funciones de media móvil o cálculo de series temporales. Controlan la "memoria a corto/largo plazo" del algoritmo.
2. RISK	RiskPercent, SL_ATR_Mult, TP_ATR_Mult	Gestión de memoria/capital. <i>Stop Loss (SL)</i> es tu bloque try-catch financiero: corta la operación si hay fallo crítico. <i>Take Profit (TP)</i> es la condición de salida exitosa.
3. TRADE MANAGEMENT	UseBreakEven, UseTrailing	Funciones de protección activa. Un <i>Trailing Stop</i> ajusta dinámicamente el límite de pérdidas a medida que la ganancia sube, asegurando la rentabilidad (Garbage collection de pérdidas).

4. FILTERS & SYSTEM

TradingHours,
MagicNum

Condicionales de ejecución (Ej: if time > 01:00). El *MagicNumber* es el Process ID (PID) de tus operaciones, crucial para que el bot identifique qué órdenes son tuyas y no modifique las del usuario.

Elija un asesor experto...

Visualizar los resultados de las simulaciones anteriores

Variable	Valor	Empezar	Paso	Parar	Pasos
1. STRATEGY (GOLD TREND) ===					
☒ Inp_FastMA	10	10	1	100	
☒ High Period Filter	50	50	1	500	
☒ Inp_RSI_Period	14	14	1	140	
☒ Inp_RSI_Buy_Lvl	50.0	50.0	5.0	500.0	
☒ Inp_RSI_Sell_Lvl	50.0	50.0	5.0	500.0	
☒ Inp_ATR_Period	14	14	1	140	
2. RISK (SAFE 0.25%) ===					
☒ Ultra Safe	0.25	0.25	0.025	2.5	
☒ Wide SL (Breathing room)	2.0	2.0	0.2	20.0	
☒ Short TP (Secure bag)	1.5	1.5	0.15	15.0	
☒ Inp_MaxPosSymbol	1	1	1	10	
☒ Inp_MaxPosTotal	3	3	1	30	
3. TRADE MANAGEMENT ===					
☒ Inp_UseBreakEven	true	false		true	
☒ Wait for 1 ATR	1.0	1.0	0.1	10.0	
☒ +20 Points	20.0	20.0	2.0	200.0	
☒ Inp_UseTrailing	true	false		true	
☒ Inp_Trail_Start	1.5	1.5	0.15	15.0	
☒ Inp_Trail_Step	0.5	0.5	0.05	5.0	
4. FILTERS ===					
☒ Inp_TradingHours	01:00-23:00				
☒ Gold spread varies	1000000	1000000	1	10000000	
☒ Inp_Slippage	10	10	1	100	
☒ Inp_MagicNum	20250103	20250103	1	202501030	
5. SYSTEM ===					
☒ Inp_EnableCSV	true	false		true	
☒ Inp_DrawVisuals	true	false		true	

General

Configuración

Parámetros de entrada

Backtest

Gráfico

Agentes

Diario

Empezar

Escenario 7: Resultados del Backtest (Model Evaluation Metrics)

Una vez simulado el mercado pasado, MetaTrader devuelve un informe. Aquí es donde decides si el modelo de Machine Learning es lo suficientemente robusto para ir a producción.

 **Traducción CS:** Este es el **Performance Report** de la fase de testing. Evalúa la eficacia, eficiencia y tolerancia a fallos de tu modelo predictivo.

Calidad del historial (History Quality)

Porcentaje de datos sin errores ni huecos durante la prueba.

 **CS:** Data Integrity Check. Si es menor al 99%, tus datos (Dataset) tienen valores nulos (NaN) o corruptos. Las predicciones del modelo no serán fiables.

Beneficio Neto vs Reducción Máxima (Drawdown)

Beneficio Neto: Ganancia total. *Reducción Máxima:* La caída más fuerte que sufrió la cuenta desde un pico alto hasta un valle.

 **CS:** El *Drawdown* es tu peor escenario de falla (Worst-case scenario / System Degradation). Un sistema que gana mucho pero tiene un Drawdown del 50% es como una app rápida que crashea el servidor la mitad del tiempo.

Factor de Beneficio (Profit Factor)

Se calcula dividiendo el Beneficio Bruto entre la Pérdida Bruta.

 **CS: Ratio de Eficiencia.** Un valor mayor a 1.0 indica que el sistema gana más de lo que pierde. Los Quants buscan valores > 1.5 para considerar un modelo robusto.

Ratio de Sharpe (Sharpe Ratio)

Mide cuánto beneficio genera la estrategia por cada unidad de riesgo (volatilidad) asumida.

 **CS: Performance por Watt.** No basta con que el algoritmo gane dinero; debe hacerlo con la menor fluctuación (ruido) posible. Un Sharpe > 1.0 es bueno, > 2.0 es excelente.

